S.G. Ganesh

Demystifying the 'Volatile' Keyword in C

Most programmers don't understand the meaning and significance of the 'volatile' keyword. So let's explore that this month.

One of my favourite interview questions for novice programmers is: "What is the use of the 'volatile' keyword?" For experienced programmers, I ask: "Can we qualify a variable as both 'const' and 'volatile'—if so, what is its meaning?" I bet most of you don't know the answer, right?

The keyword 'volatile' is to do with compiler optimisation. Consider the following code:

```
long *timer = 0x0000ABCD;
// assume that at location 0x0000ABCD the current time is available
long curr_time = *timer;
// initialize curr_time to value from 'timer'
// wait in while for 1 sec (i.e. 1000 millisec)
while( (curr_time - *timer) < 1000 )
{
   curr_time = *timer; // update current time
}
print_time(curr_time);
// this function prints the current time from the
// passed long variable
```

Usually, hardware has a timer that can be accessed from a memory location. Here, assume that it's 0x0000ABCD and is accessed using a long * variable 'timer' (in the UNIX tradition, time can be represented as a long variable and increments are done in milliseconds). The loop is meant to wait one second (or 1,000 milliseconds) by repeatedly updating curr_time with the new value from the timer. After a one second delay, the program prints the new time. Looks fine, right?

However, from the compiler point of view, what the loop does is stupid—it repeatedly assigns curr_time with *timer, which is equivalent to doing it once outside the loop. Also, the variable 'timer' is de-referenced repeatedly in the loop—when it is enough to do it once. So, to make the code more efficient (i.e., to optimise it), it may modify loop code as follows:

```
curr_time = *timer; // update current time
long temp_time = *timer;
while( (curr_time - temp_timer) < 1000 )
{  /* do nothing here */
}
```

As you can see, the result of this transformation is disastrous: the loop will never terminate because neither is curr_time updated nor is the timer de-referenced repeatedly to get new (updated time) values.

What we need is a way to tell the compiler not to 'play around' with such variables by declaring them volatile, as in:

```
volatile long * timer = 0x0000ABCD;
volatile curr_time = *timer;
```

Now, the compiler will not do any optimisation on these variables. This, essentially, is the meaning of the 'volatile' keyword: It declares the variables as 'asynchronous' variables, i.e., variables that are 'not-modified-sequentially'. Implicitly, all variables that are not declared volatile are 'synchronous variables'.

How about qualifying a variable as both *const* and volatile? As we know, when we declare a variable as *const*, we mean it's a 'read-only' variable—once we initialise it, we will not change it again, and will only read its value. Here is a modified version of the example:

```
long * const timer = 0x0000ABCD;
// rest of the code as it was before.
```

We will never change the address of a timer, so we can put it as a const variable. Now, remember what we did to declare the timer as volatile:

```
volatile long * timer = 0x0000ABCD;
```

We can now combine const and volatile together:

```
volatile long * const timer = 0x0000ABCD;
```

It reads as follows: the timer is a *const* pointer to a long volatile variable. In plain English, it means that the timer is a variable that I will not change; it points to a value that can be changed without the knowledge of the compiler! END

About the author:

S G Ganesh is a research engineer in Siemens (Corporate Technology). His latest book is "60 Tips on Object Oriented Programming", published by Tata McGraw-Hill. You can reach him at sgganesh@gmail.com.